

Library Management System

A Data-Structure-Driven Desktop Application in C++17

Key Highlights

- **Custom data structures:** BST (Books), Singly Linked List (Students), Circular Queue (Requests), Circular History buffer (Transactions)
- **GUI-first:** Dear ImGui + SFML backend (professional desktop workflow)
- **Persistence:** CSV-based storage with backups and safer saves
- **Quality:** Unit + integration + performance tests (CTest)

Project Manager: Suliman Naseeri

Team: Muhammad Asif Khan, Nelorfar Hussain, Muneera Omer, Momina Ali, Muhammad Yousaf, Dawood Shah, Hammad Durrani

Language: C++17

GUI Stack: Dear ImGui + ImGui-SFML + SFML

Build System: CMake

Date: January 8, 2026

Contents

1	Overview	2
2	System Architecture	2
2.1	High-Level Components	2
2.2	Architecture Diagram	3
3	Data Structures	3
3.1	BST for Books	3
3.2	Singly Linked List for Students	3
3.3	Circular Queue for Book Requests	3
3.4	Transaction History (Circular Buffer)	3
4	Core Workflows	4
4.1	Book Management	4
4.2	Student Management	4
4.3	Issue / Return	4
4.4	Authentication and Permissions	4
5	Persistence (CSV + Backups)	4
5.1	CSV Format Notes	4
5.2	Backups	5
6	Graphical User Interface (Dear ImGui + SFML)	5
6.1	Design Goals	5
6.2	Screens	5
6.3	Screenshots	5
7	Build, Run, and Test	5
7.1	Build (GUI)	5
7.2	Login (Demo Credentials)	5
8	Team Contributions (8 Members)	5
8.1	Tests	6
8.2	Sanitizers (Debug hardening)	6
9	Performance Notes	6
10	Conclusion and Future Work	7
10.1	Future Enhancements	7

1 Overview

Library Management System is a complete library workflow system focused on correctness, performance, and a professional GUI. It supports core library operations: managing books and students, issuing/returning books with validation, request queues when books are unavailable, and transaction history tracking.

New in the Final Version

- **Two-mode login:** Student vs Librarian credentials at startup.
- **Role-based permissions:** Librarian has full CRUD and issue privileges; Students have limited operations.
- **Student Assistant (offline):** chat-style helper for guidance and book recommendations.
- **Safe demo seeding:** adds 2 demo students + extra books only if missing (avoids duplicate ID spam).

Goals

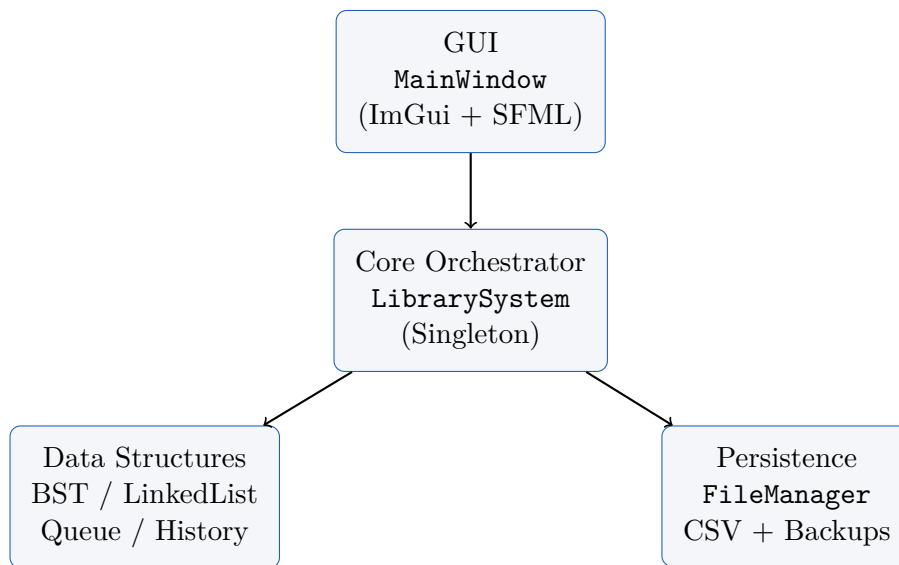
- Provide **fast book search** using a BST keyed by book ID.
- Enforce **student borrowing rules** (3-book limit) at the domain level.
- Maintain **request queues** per book (FIFO with priority input).
- Persist data reliably with **CSV** files and **backup/restore**.
- Deliver a **GUI-first** user experience using Dear ImGui + SFML.

2 System Architecture

2.1 High-Level Components

- **Core domain:** Book, Student, Transaction, BookRequest
- **Data structures:** BST<T>, LinkedList<T>, ArrayQueue<T>, TransactionHistory
- **Coordinator:** LibrarySystem (singleton)
- **Persistence:** FileManager (CSV + backups)
- **Presentation:** MainWindow (Dear ImGui + SFML)

2.2 Architecture Diagram



3 Data Structures

3.1 BST for Books

Why BST? Book search by ID is frequent and must be efficient. A BST supports average-case $O(\log n)$ insert/search/remove.

- Key: `Book::id`
- Operations: `insert`, `search(ID)`, `update`, `remove`, traversals, range search
- Thread-safety: read-write locking via `std::shared_mutex`

3.2 Singly Linked List for Students

Students are stored in a linked list:

- Efficient append, simple iteration, and straightforward serialization.
- Thread-safety via `std::mutex`.

3.3 Circular Queue for Book Requests

Requests are stored using a circular queue implementation:

- FIFO ordering per book.
- Operations: `enqueue`, `dequeue`, `cancel`, `find position`.

3.4 Transaction History (Circular Buffer)

Transaction history keeps the most recent N transactions in a fixed-size circular structure:

- Fast $O(1)$ insertion
- Supports “recent transactions” and student/book filtering

4 Core Workflows

4.1 Book Management

- Add book (validation: ID range, non-empty title/author, ISBN-13, quantity ≥ 1)
- Search by ID (BST), or by Title/Author (linear scan)
- Update book info / quantity
- Remove book (guarded by system rules)

4.2 Student Management

- Register student (6-digit ID, name/department required, email validated if provided)
- Borrowing limit enforced (max 3 active borrows)
- Search by ID / name

4.3 Issue / Return

- Issue (Librarian only): verify student exists and can borrow; verify book availability; create transaction; update counts
- Return: update book availability; mark transaction returned; compute fine if overdue
- When a book becomes available, the system can process waiting requests (queue)

4.4 Authentication and Permissions

Login modes:

- **Librarian** (admin role): full management privileges.
- **Student**: limited actions, focused on returning and requesting books.

Policy:

- Librarian can: Add/Edit/Delete books and students; Issue books; Return books; Save/Reload.
- Student can: Search/view books; Request books; Return books (for their own ID); use Assistant tab.

5 Persistence (CSV + Backups)

Data is stored under `data/` in CSV form:

- `books.csv`
- `students.csv`
- `transactions.csv`
- `requests.csv`

5.1 CSV Format Notes

The loader supports:

- Header lines (skipped)
- Backward compatibility for older `books.csv` rows
- Best-effort loading (bad rows are skipped rather than crashing the app)

5.2 Backups

Before destructive operations and major saves, backups are created in `data/backups/`. This protects against accidental deletes or partial writes.

6 Graphical User Interface (Dear ImGui + SFML)

6.1 Design Goals

- Desktop-app layout: sidebar navigation, content area, status bar
- Fast workflows: searchable tables, edit/delete modals, toasts for feedback
- Pleasant visuals: light themes with configurable accents

6.2 Screens

- Login: choose Student/Librarian mode and authenticate
- Dashboard: overview cards and recent activity preview
- Books: search, list, add, edit, delete
- Students: search, list, register, delete
- Issue/Return: quick forms with feedback
- Requests: per-book queues
- History/Statistics: tables + dashboard summaries
- Student Assistant: offline help + recommendations (Student role)

6.3 Screenshots

7 Build, Run, and Test

7.1 Build (GUI)

```
1 cmake -S . -B build-gui -DCMAKE_BUILD_TYPE=Release
2 cmake --build build-gui -j 4
3 ./build-gui/library_gui
```

7.2 Login (Demo Credentials)

- Librarian: `admin` / `admin123`
- Student: `100001` / `0001`, `100002` / `0002`
PIN rule: last 4 digits of the Student ID (e.g., `100002` → `0002`).

8 Team Contributions (8 Members)

Distribution of Work

Project Manager: Suliman Naseeri

Critical sections assigned to: Muhammad Asif Khan

Member	Responsibilities
Suliman Naseeri (PM)	Planning, task breakdown, milestones, reviews, coordination, final submission checklist.
Muhammad Asif Khan (Critical)	Core integration and correctness: LibrarySystem orchestration, GUI integration, login/roles enforcement, persistence reliability, final build hygiene.
Nelorfur Hussain	UI/UX review and consistency: layout polish, theme coherence, accessibility/labels, screenshot preparation.
Muneera Omer	Documentation lead: LaTeX report structure, diagrams/flow descriptions, update notes, ensuring docs match implementation.
Momina Ali	Testing and QA: unit/integration tests, edge cases (borrow limits, duplicates), regression checks after changes.
Muhammad Yousaf	Data structures review: BST/LinkedList/Queue correctness, complexity notes, test scenarios.
Dawood Shah	File I/O and data formats: CSV structure verification, backup/restore behavior, data migration/backward-compat checks.
Hammad Durrani	Student assistant content and recommendations: refining prompts/responses, improving recommendation heuristics, validating student-only permissions.

8.1 Tests

Tests are built as small executables and run via CTest:

```

1 cmake -S . -B build-tests -DCMAKE_BUILD_TYPE=Debug -DBUILD_TESTS=ON
2 cmake --build build-tests -j 4
3 ctest --test-dir build-tests --output-on-failure

```

8.2 Sanitizers (Debug hardening)

```

1 cmake -S . -B build-asan -DCMAKE_BUILD_TYPE=Debug -DBUILD_TESTS=ON -
  DENABLE_SANITIZERS=ON
2 cmake --build build-asan -j 4
3 ctest --test-dir build-asan --output-on-failure

```

9 Performance Notes

Component	Notes
BST (Books)	Average-case $O(\log n)$ for insert/search/remove; inorder traversal provides sorted listing.
LinkedList (Students)	Linear search $O(n)$, acceptable for moderate student counts.
Queue (Requests)	$O(1)$ enqueue/dequeue; supports cancellation/position lookups.
Transaction History	Fixed-size circular buffer with $O(1)$ add and fast “recent” queries.

10 Conclusion and Future Work

This project demonstrates a full end-to-end system: strong domain validation, custom data structures, persistence, and a professional GUI.

10.1 Future Enhancements

- Richer search (filters, sorting persistence, advanced query UI)
- Better reporting (export to PDF/CSV, charts)
- Stronger request processing policies (true priority queue)
- Improved configuration system (editable `settings.ini` driving runtime behavior)

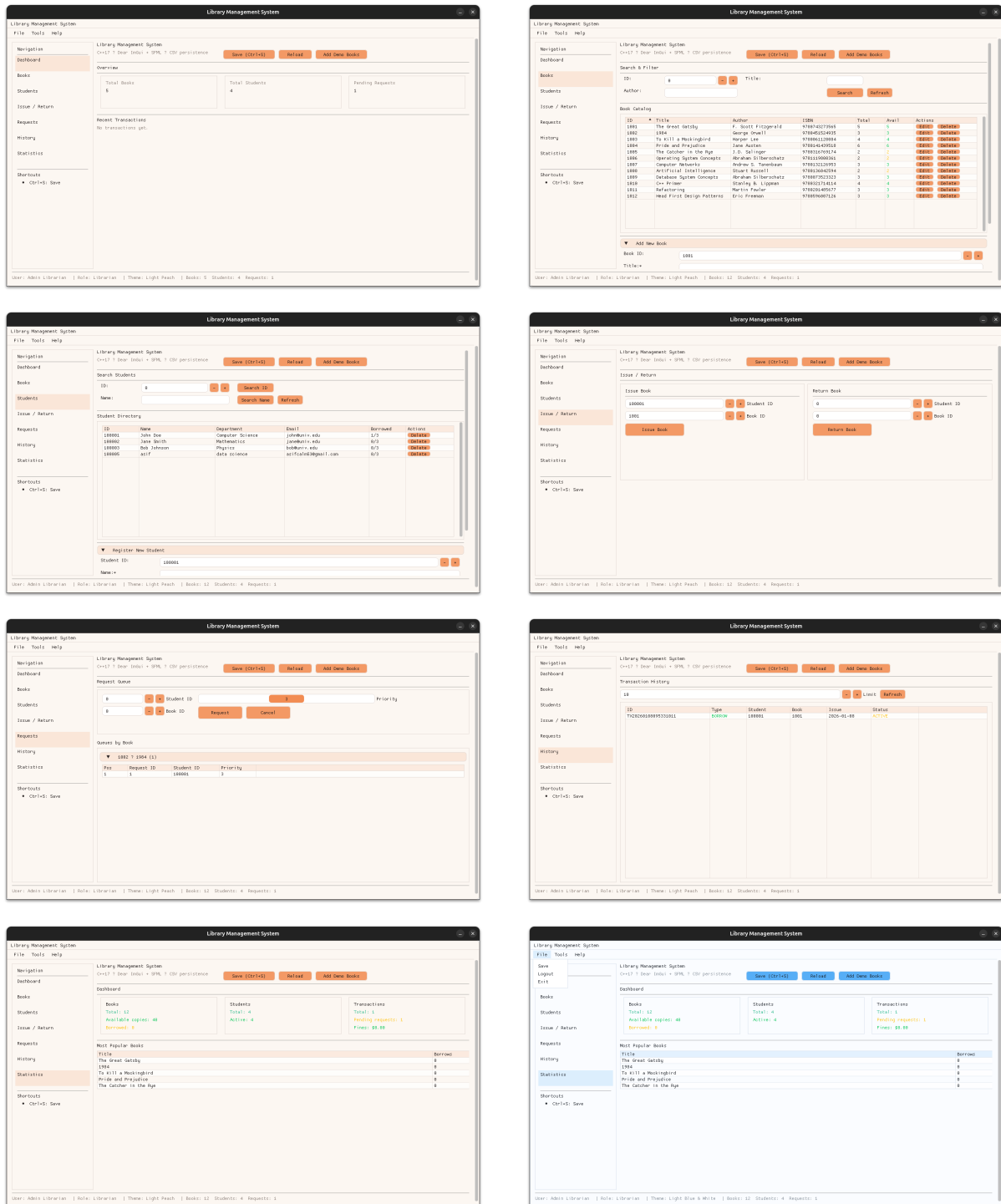


Figure 1: GUI screenshots: Dashboard, Books, Students, Issue/Return, Requests, History, and Statistics.